



Examen MU4IN505 CPA. Durée: 2h.

BM. Bui-Xuan

Tout document personnel non-électronique autorisé. Toute réponse sans preuve aura valeur nulle.

Exercice 1. Estimation du temps de calcul (2 points)

Supposons que nous avons à exécuter sur un ordinateur un processus prenant en entrée un argument de taille $\approx n$ B (byte). Supposons également que l'unité de calcul (CPU) de cet ordinateur effectue ≈ 1 G (giga) instructions par seconde. Nous assumons que 1B est la plus petite unité accessible par ordinateur à chaque instruction.

Q1. L'analyse théorique de ce processus prouve que sa complexité est de l'ordre de $\Theta(n^2)$. Estimer le temps de calcul de ce processus pour:

- $n = 1$ K (kilo); $n = 10000$ (dix mille); $n = 1$ M (mega); $n = 1$ G (giga); $n = 1$ T (tera).

Q2. Donner les mêmes (cinq) estimations en supposant que la complexité théorique du processus est en $\Theta(n \log n)$.

N.B.: par ordre de grandeur, un processus de complexité $\Theta(n^2)$ aura à effectuer:

- $(10^3)^2$ instructions en prenant en entrée un argument de taille $1\text{KB} = 10^3\text{B}$.
- $(10^4)^2$ instructions en prenant en entrée un argument de taille 10000B .
- $(10^6)^2$ instructions en prenant en entrée un argument de taille $1\text{MB} = 10^6\text{B}$.
- $(10^9)^2$ instructions en prenant en entrée un argument de taille $1\text{GB} = 10^9\text{B}$.
- $(10^{12})^2$ instructions en prenant en entrée un argument de taille $1\text{TB} = 10^{12}\text{B}$.

Le CPU pouvant effectuer $\approx 10^9 = 1\text{G}$ instructions par seconde, le temps de calcul de ce processus sera:

- ≈ 1 milli seconde pour le cas $n = 1\text{K}$ (kilo);
- ≈ 100 milli secondes pour le cas $n = 10000$ (dix mille);
- $\approx \frac{1}{4}h \approx 1000$ secondes pour le cas $n = 1\text{M}$ (mega);
- ≈ 30 ans $\approx 10^9$ secondes pour le cas $n = 1\text{G}$ (giga);
- ≈ 30 million d'années $\approx 10^{24}$ secondes pour le cas $n = 1\text{T}$ (tera).

Si la complexité du processus est $\Theta(n \log n)$, le nombre d'instructions:

- pour $n = 1\text{K}$ (kilo) est $10^3 \log 10^3 = 10^3 \log 2^{10} = 10^3 \cdot 10 = 10^4$; le temps est ≈ 10 micro secondes;
- pour $n = 10000$ (dix mille) est $13 \cdot 10^4 \approx 10^5$; le temps est ≈ 100 micro secondes;
- pour $n = 1\text{M}$ (mega) est $2 \cdot 10^7$; le temps est ≈ 20 milli secondes;
- pour $n = 1\text{G}$ (giga) est $3 \cdot 10^{10}$; le temps est ≈ 30 secondes;
- pour $n = 1\text{T}$ (tera) est $4 \cdot 10^{13}$; le temps est $\approx 1h \approx 4000$ secondes.

Exercice 2. Enveloppe convexe (6 points)

D'un point de vue de structure de données en 2D, un *polygone* est une liste ordonnée de points dans le plan, où l'ordre des points suit le sens trigonométrique du repère cartésien (c.à.d. de l'axe $\vec{Ox}$ vers l'axe $\vec{Oy}$). Avec cette structure de donnée, on appelle *côté* du polygone tout segment reliant deux points consécutifs dans la liste définissant le polygone. Un polygone est *simple* si la seule intersection possible entre deux côtés quelconques du polygone est l'un des extrémités de ces deux côtés.

Par abus de langage, on confond le polygone et la partie du plan situé à l'intérieur des segments délimitant le polygone. Un polygone est *convexe* s'il est simple et s'il contient tout segment reliant deux points à l'intérieur du polygone. L'*enveloppe convexe* d'un objet géométrique (liste de points, liste de segments, polygone, ...) est le plus petit polygone convexe contenant cet objet. Dans les questions suivantes, les seuls langages de programmation autorisés sont: java, python et C.

- Q1.** (Question de cours) Donner le code d'une fonction qui prend en entrée P un polygone simple comprenant n points et qui retourne en temps $O(n)$ l'enveloppe convexe de P . (On peut penser au filtre Graham.)
- Q2.** (Diviser-pour-régner) Donner le code d'une fonction qui prend en entrée deux polygones convexes P_1, P_2 comprenant au total n points et qui retourne en temps $O(n)$ l'enveloppe convexe de $P_1 \cup P_2$.
- Q3.** (Solution en temps $O(n \log n)$) Utiliser la précédente implémentation et donner le code d'une fonction récursive qui prend en entrée une liste quelconque de n points et qui retourne en temps $O(n \log n)$ l'enveloppe convexe de ces points. *N.B.*: il est impérative que la fonction soit récursive, et que le paradigme algorithmique soit de type diviser-pour-régner, comme par exemple la dichotomie.
- Q4.** (Optimalité) Nous supposons dans cette question avoir accès à une fonction `enveloppeConvexe` prenant en entrée n points dans le plan et retournant en temps τ l'enveloppe convexe de ces points. Donner le code d'une fonction qui prend en entrée une liste quelconque de n entiers et qui retourne en temps $O(n + \tau)$ ces entiers triés par ordre croissant. Rappeler la complexité la plus optimale pour le tri d'une liste de n entiers. Donner alors la complexité la plus optimale d'un algorithme calculant l'enveloppe convexe d'un ensemble de n points dans le plan.

Rappel important: pour qu'une réponse soit considérée, il est impératif de la justifier...

N.B.: les idées principales sont les suivantes:

- Q1.** Les points du polygone étant déjà ordonnés en suivant le sens trigonométrique, ce qui reste à faire pour calculer l'enveloppe convexe est d'effectuer le filtre de Graham sur ces points. Effectuer les opérations suivantes pour tout triplet p, q, r de points successifs du polygone. Si la valeur du produit vectoriel $\vec{pq} \wedge \vec{pr}$ est positive, marquer p, q, r comme "DONE", sinon, enlever le point q du polygone, reculer d'une position, puis répéter le traitement pour le triplet suivant jusqu'à ce que tous les triplets sont marqués "DONE".
- Q2.** Nous allons adapter l'algorithme Jarvis pour fusionner P_1 et P_2 en temps linéaire. Sans perte de généralité supposons parmi les points $P_1 \cup P_2$ que le point le plus à l'Ouest (formellement le point d'abscisse minimum, et en cas d'ex aequo, celui d'ordonnée minimum) appartient à P_1 . Initialiser un point q ayant pour valeur le point le plus à l'Ouest. Initialiser deux points p, r_1 ayant pour valeurs respectives les points précédant et succédant le point q selon l'ordre trigonométrique dans P_1 . Initialiser un point r_2 ayant pour valeur le point de P_2 tel que l'angle entre $\vec{pq}$ et $\vec{qr}_2$ est minimum. Tant qu'il reste des points dans $P_1 \cup P_2$ qui ne sont pas marqués par "DONE", effectuer les opérations suivantes. Si l'angle $(\vec{pq}, \vec{qr}_1)$ est plus petit que l'angle $(\vec{pq}, \vec{qr}_2)$, marquer q comme "DONE" et effectuer les mises à jours suivantes: $p \leftarrow q$; $q \leftarrow r_1$; $r_1 \leftarrow$ le point suivant r_1 selon l'ordre trigonométrique dans P_1 ; dans P_2 avancer r_2 au point tel que l'angle $(\vec{pq}, \vec{qr}_2)$ est minimum, tout en enlevant les points de P_2 se situant entre l'ancienne valeur et la nouvelle valeur de r_2 . Si les deux angles sont égaux et la distance $|qr_1|$ est au plus $|qr_2|$, effectuer les mêmes opérations que précédemment. Dans le restant des cas, marquer q comme "DONE" et effectuer les mises à jours

suivantes: $p \leftarrow q$; $q \leftarrow r_2$; $r_2 \leftarrow$ le point suivant r_2 selon l'ordre trigonométrique dans P_2 ; dans P_1 avancer r_1 au point tel que l'angle $(\vec{pq}, \vec{qr}_1)$ est minimum, tout en enlevant les points de P_1 se situant entre l'ancienne valeur et la nouvelle valeur de r_1 ; enfin, inverser P_1 et P_2 , ainsi qu'inverser r_1 et r_2 .

- Q3.** Nous allons nous inspirer de l'algorithme *TriFusion* pour calculer l'enveloppe convexe d'un polygone P contenant n points en temps $O(n \log n)$. Si P contient moins de 3 points retourner $E = P$, l'enveloppe convexe de P . Sinon, partitionner P en deux listes $P = P_1 \uplus P_2$ de taille égale, à une unité près: $-1 \leq |P_1| - |P_2| \leq 1$. Effectuer des appels récursifs et calculer E_1 et E_2 , les enveloppes convexes respectifs de P_1 et P_2 . Appeler la fonction décrite dans la question précédente pour fusionner les deux enveloppes E_1 et E_2 en E , l'enveloppe convexe de $E_1 \cup E_2$. Retourner E comme enveloppe convexe de P .
- Q4.** La complexité optimale pour trier une liste $L = [x_1, x_2, \dots, x_n]$ contenant n entiers est $\Theta(n \log n)$. Après le calcul de E , l'enveloppe convexe de la liste de points $P = [(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)]$, il est possible de parcourir E en commençant par le point le plus à l'Ouest de E et retourner la liste de toutes les abscisses des points de E . La liste retournée est la liste L , triée par ordre croissant. Donc, la complexité optimale pour calculer l'enveloppe convexe d'une liste de n points est $\Theta(n \log n)$.

Exercice 3. Tri paniers (6 points)

Le problème de suppression de duplicata consiste à, étant donnée une liste de n entiers, calculer une sous-liste de celle-ci, où chaque entier initial apparaît une et une seule fois dans la sous-liste calculée.

- Q1.** Donner un algorithme naïf résolvant la suppression de duplicata d'une liste de n entiers en temps $O(n^2)$.
- Q2.** (Stream) Afin d'accélérer ce calcul, nous proposons d'utiliser l'approche fonctionnelle suivante, où le tableau `Panier` contient des bits tous initialisés à 0. Par convention, nous assimilons le bit 0 à la valeur `FALSE`, et tout bit différent de 0 à la valeur `TRUE`. Nous appelons X la taille initiale du tableau `Panier, qui sera à déterminer dans ce qui suit.`

$\lambda : x \mapsto \text{if}(!\text{Panier}[x])\{\text{print } x; \text{Panier}[x]++;\}$

En utilisant la fonction λ , est-il possible de résoudre correctement la suppression de duplicata d'une liste L contenant n entiers en temps $O(n + X)$, où X est un paramètre de L , et où, selon les instances nous pouvons tout aussi bien avoir $X > n^n$ comme $n > X^X$? Si oui, donner le pseudo-code des étapes principales d'un tel algorithme, ainsi que sa complexité. Prouver formellement cette complexité, soit par analyse directe, soit par induction.

Proposer une valeur de X exprimée en fonction des éléments de la liste L . Quelle est l'ordre de grandeur de $n + X$, lorsque $n = 10000$ (dix mille) et lorsque le nombre de bits des entiers de la liste L à trier sont inférieur à 10? à 20? à 30? à 40? à 64? à 512? Nous rappelons qu'un entier de 10 bits est compris entre 0 et 1023.

- Q3.** (Stream + hash) Nous assumons qu'il est possible de définir un processus qui, prenant en entrée un entier x , retourne en temps constant¹ la valeur $f(x)$ d'une fonction préalablement fixée $f : \mathbb{N} \mapsto \llbracket 0, 2^{512} \rrbracket$. Est-il possible de résoudre correctement la suppression de duplicata d'une liste de n entiers en temps $O(n + 2^{512})$? Si oui, donner le pseudo-code des étapes principales d'un tel algorithme. Prouver formellement cette complexité, soit par analyse directe, soit par induction.
- Q4.** (Stream + hash + addressing) Nous remarquons que 2^{512} est un facteur bloquant dans une utilisation pratique: ce chiffre dépasse l'estimation du nombre d'atomes dans l'univers ($\approx 10^{82}$). Nous supposons maintenant qu'il est possible de lire une suite de 64 adresses-mémoire en temps constant (ici, 64

¹Nous pouvons citer dans ce sens les fonctions SHA-512, MD6, BLAKE-512, SWIFFT, ..., dont le temps de calcul est considéré en condition réelle comme quasi-constante. De plus, dans de nombreux cas particuliers, d'autres fonctions non-cryptographiques peuvent être utilisées afin de réduire encore plus ce temps de calcul.

adresses-mémoire totalisent exactement $64 * 8 = 512$ bits, en supposant qu'une adresse-mémoire contient 1 byte = 8 bits). En s'inspirant d'une structure de liste doublement chaînée, expliquer par un dessin et par pseudo-code les étapes principales d'un algorithme de suppression de duplicata en temps quasi-linéaire².

- Q5.** Pourrions nous appliquer les mêmes procédures pour trier une liste de n entiers en temps quasi-linéaire?

N.B.: les idées principales sont les suivantes:

- Q1.** Appelons L la liste à traiter. Pour tout élément e dans la liste L , reparcourir la liste L depuis le début jusqu'à la position e , s'il y a un élément $e' = e$ lors du second parcours, enlever e' de la liste L .
- Q2.** Avec $X = \max L$, nous pouvons initialiser *Panier* comme un tableau ayant $X + 1$ entrées, puis, appliquer λ sur la liste L . La complexité de cette procédure est $O(X)$ pour l'initialisation du tableau *Panier*, et $O(n)$ pour appliquer la fonction λ sur tous les éléments de L . Si L ne contient pas d'entiers de plus de 10 bits, $X \leq 2^{10} \approx 1000$. Si $n = 10000$ nous aurons $n + X \approx n$. Pour le même n avec L contenant des entiers avec 60 bits, nous aurons $X \approx 2^{60} \approx 10^{18}$, ainsi $n + X \approx X$.
- Q3.** Idem que précédemment en remplaçant la fonction λ par λ_f , définie par:

$$\lambda_f : x \mapsto \text{if}(!\text{Panier}[f(x)])\{\text{print } x; \text{Panier}[f(x)]++;\}$$
- Q4.** A la place d'initialiser un tableau *Panier* avec autant d'entrées que X , nous pouvons initialiser un tableau doublement chaîné *Addresses* pour stocker toute nouvelle valeur $f(x)$ utilisée pendant l'application de la fonction λ_f à la liste L . Si l'accès à chaque index $f(x)$ stocké dans *Addresses* est en temps constant, l'application de la fonction λ_f à la liste L sera en $O(n)$.
- Q5.** La réponse rapide est non parce que la complexité optimale du tri est $\Theta(n \log n)$. Une autre réponse serait de dire qu'en cas de l'utilisation du tableau *Panier* à trier, après l'application de la fonction λ à la liste L , il reste à reparcourir tous les entrées de *Panier* pour sortir les éléments x par ordre croissant, ce qui réintroduit le facteur $O(X)$. Si on utilise λ_f à la place, comme f n'est pas nécessairement une fonction croissante, l'ordre des $f(x)$ n'implique pas forcément l'ordre des x . Enfin, si on remplaçait le tableau *Panier* par le tableau doublement chaîné *Addresses*, ce dernier ne trie pas ses index.

Exercice 4. Programmation dynamique (6 points)

- Q1.** Nous considérons la représentation d'un graphe pondéré G par une matrice M_G : $M_G(i, j)$ a pour valeur ∞ s'il n'y a pas d'arête entre les sommets i et j ; $M_G(i, j)$ a pour valeur le poids de l'arête dans le cas contraire. Nous considérons de plus que le poids de l'arête représente le coût pour traverser l'arête (i, j) . Une méthode naïve pour calculer la matrice de distance M_{dist} , où $M_{dist}(i, j)$ a pour valeur la distance géodésique (i.e. celle dans le graphe G) entre les sommets i et j est de définir M^p , la matrice où $M^p(i, j)$ est la distance entre i et j des chemins utilisant au plus p sommets dans G , pour $2 \leq p \leq n$ et n le nombre de sommets dans G . Nous remarquons la formule de récurrence suivante:

$$M^{p+1}(i, j) = \min_k M^p(i, k) + M^2(k, j).$$

Proposer un algorithme calculant M_{dist} en passant par le calcul de M^p , pour $2 \leq p \leq n$. Quelle est sa complexité?

- Q2.** (Algorithme Floyd-Warshall) Soit N^p la matrice où $N^p(i, j)$ est la distance entre i et j des chemins passant uniquement par les p premiers sommets de G . Donner une formule de récurrence pour le calcul de N^{p+1} à partir de la connaissance de N^p . En déduire un calcul plus rapide de M_{dist} en passant par le calcul de N^p . Quelle est sa complexité?

²Nous pouvons faire l'expérience sur n'importe quel console avec la commande `awk '!Panier[$0]++' nomfichier`.

Q3. Soit un ensemble de n objets distincts dont chacun a respectivement valeur v_i et poids w_i , pour $1 \leq i \leq n$. De plus, nous supposons que v_i et w_i sont des entiers positifs, pour $1 \leq i \leq n$. Soit W un entier positif représentant le poids maximum qu'une valise peut contenir. Nous voudrions calculer la somme maximum S , en terme de somme des valeurs v_i , que cette valise peut contenir. Plus formellement, nous voudrions calculer

$$S = \max \left(\sum_{i=1}^n x_i \times v_i \right)$$

tel que

$$\sum_{i=1}^n x_i \times w_i \leq W \text{ et } x_i \in \{0, 1\} \text{ pour tout } 1 \leq i \leq n.$$

Nous définissons le vecteur M^i où $M^i(w)$ est la somme maximum des valeurs qu'une valise de poids maximum w peut contenir en utilisant uniquement les i premiers objets parmi les n objets initiaux. Nous remarquons la formule de récurrence suivante: si $w_{i+1} > w$ alors $M^{i+1}(w) = M^i(w)$, sinon,

$$M^{i+1}(w) = \max(M^i(w), v_{i+1} + M^i(w - w_{i+1})).$$

Proposer un algorithme calculant la valeur S . Quelle est sa complexité?

N.B.: les idées principales sont les suivantes:

- Q1.** Initialiser deux matrices M et M_0 ayant toutes deux pour valeur la matrice M^2 . Pour tout $2 \leq p < n$, effectuer deux étapes: (1) $M_{prev} \leftarrow M$ et (2) pour toute colonne i de M , pour toute ligne j de M , effectuer trois sous-étapes: (2a) $minvaleur \leftarrow \infty$, puis (2b) pour tout k , si $M_{prev}[i][k] + M_0[k][j] < minvaleur$ alors $minvaleur \leftarrow M_{prev}[i][k] + M_0[k][j]$, enfin (2c) $M[i][j] \leftarrow minvaleur$. Après les itérations, retourner M .
- Q2.** La formule de récurrence pour $2 \leq p < n$ est $N^{p+1}(i, j) = \min(N^p(i, j), N^p(i, p+1) + N^p(p+1, j))$. Le calcul peut être le suivant. Initialiser une matrice N ayant pour valeur la matrice M^2 . Pour tout $2 \leq p < n$, pour toute colonne i de N , pour toute ligne j de N , si $N[i][p+1] + N[p+1][j] < N[i][j]$ alors $N[i][j] \leftarrow N[i][p+1] + N[p+1][j]$. Après les itérations, retourner N .
- Q3.** Initialiser une matrice M ayant pour valeur la matrice M^1 . Pour tout $1 \leq i < n$, pour tout index $w \geq w_{i+1}$ de M , si $v_{i+1} + M[w - w_{i+1}] > M[w]$ alors $M[w] \leftarrow v_{i+1} + M[w - w_{i+1}]$. Après les itérations, retourner M .